PYTHON PROGRAMMING

Complete 2-Month Course Material | Beginner to Intermediate

8 weeks | 40 class days | 80 guided hours | 8 modules | Exercises, quizzes, projects and solutions

Course outcomes

Build programs using variables, decisions, loops, collections, functions, files, OOP, databases, and HTTP APIs. Complete a tested command-line project and explain its design.

Suggested daily schedule

20 min recap and concept discussion; 35 min guided coding; 45 min hands-on practice; 15 min review; 5 min exit quiz. Homework adds optional independent practice.

Week 1: Python Foundations

Learning sequence: concepts, guided code, independent lab, daily review.

Day 1 - Setup and first program

Concepts: Install Python 3 and VS Code; use the terminal, python --version, interactive shell, scripts, indentation, and comments.

```
print("Hello, Python!")
```

Hands-on exercise: Print your name, course, and three goals.

Day 2 - Variables and types

Concepts: Identifiers, assignment, int, float, str, bool, type(), and explicit conversions.

```
age = 20
height = 1.72
print(type(age), float(age), height)
```

Hands-on exercise: Read a birth year and calculate approximate age.

Day 3 - Input and output

Concepts: input() returns text; int()/float() conversion; print(), sep/end, f-strings.

```
name = input("Name: ")
score = float(input("Score: "))
print(f"{name}: {score:.1f}")
```

Hands-on exercise: Build a student profile printer.

Day 4 - Arithmetic and expressions

Concepts: Addition, subtraction, multiplication, division, floor division, modulus, powers, precedence.

```
a, b = 17, 5
print(a / b, a // b, a % b, a ** 2)
```

Hands-on exercise: Calculate area and perimeter of a rectangle.

Day 5 - Practice and assessment

Concepts: Type conversion pitfalls, syntax errors, debugging basics and weekly recap.

```
price = 499
tax = price * 0.18
print(f"Total: {price + tax:.2f}")
```

Hands-on exercise: Mini project: shopping bill with tax and discount.

Weekly checkpoint

Complete the mini project, explain your solution, and attempt a 10-question quiz. Target: at least 70% before moving on.

Week 2: Decisions and Loops

Learning sequence: concepts, guided code, independent lab, daily review.

Day 6 - Comparison and Boolean logic

Concepts: Comparison operators, and/or/not, truth values, chained comparisons.

```
age = 19
print(age >= 18 and age < 60)
```

Hands-on exercise: Check voting eligibility and age range.

Day 7 - Conditional statements

Concepts: if, elif, else, nesting, conditional expressions.

```
mark = 76
if mark >= 90: grade = "A"
elif mark >= 70: grade = "B"
else: grade = "C"
print(grade)
```

Hands-on exercise: Build grade calculator with invalid-mark validation.

Day 8 - for loops

Concepts: range(), iteration, accumulation, enumerate().

```
total = 0
for n in range(1, 6): total += n
print(total)
```

Hands-on exercise: Print multiplication tables and sum of even numbers.

Day 9 - while loops

Concepts: Loop conditions, break, continue, counters, avoiding infinite loops.

```
n = 3
while n > 0:
    print(n)
    n -= 1
```

Hands-on exercise: Create a number-guessing game.

Day 10 - Nested loops

Concepts: Pattern printing, nested iteration and loop review.

```
for row in range(1, 5):
    print("*" * row)
```

Hands-on exercise: Mini project: menu-driven calculator.

Weekly checkpoint

Complete the mini project, explain your solution, and attempt a 10-question quiz. Target: at least 70% before moving on.

Week 3: Python Collections

Learning sequence: concepts, guided code, independent lab, daily review.

Day 11 - Lists

Concepts: Indexing, slicing, append, insert, remove, pop, sorting and iteration.

```
marks = [72, 91, 80]
marks.append(88)
print(sum(marks) / len(marks))
```

Hands-on exercise: Find highest, lowest and average marks.

Day 12 - Tuples

Concepts: Immutable sequences, unpacking, tuple returns and use cases.

```
point = (10, 20)
x, y = point
print(x, y)
```

Hands-on exercise: Represent coordinates and swap values.

Day 13 - Sets

Concepts: Unique elements, union, intersection, difference and membership.

```
a = {"python", "sql"}
b = {"python", "react"}
print(a & b)
```

Hands-on exercise: Find common skills between students.

Day 14 - Dictionaries

Concepts: Keys and values, get(), items(), update(), nested dictionaries.

```
student = {"name": "Asha", "marks": 85}
print(student.get("marks"))
```

Hands-on exercise: Create a student record lookup.

Day 15 - Collections practice

Concepts: Choosing the right collection and handling missing keys.

```
scores = {"Asha": 85, "Ravi": 72}
for name, mark in scores.items():
    print(name, mark)
```

Hands-on exercise: Mini project: student marks manager.

Weekly checkpoint

Complete the mini project, explain your solution, and attempt a 10-question quiz. Target: at least 70% before moving on.

Week 4: Functions and Modules

Learning sequence: concepts, guided code, independent lab, daily review.

Day 16 - Functions

Concepts: def, parameters, return values, docstrings and reusable design.

```
def area(width, height):
    return width * height
print(area(4, 6))
```

Hands-on exercise: Write functions for temperature conversion.

Day 17 - Arguments and scope

Concepts: Positional, keyword, default arguments; local vs global variables.

```
def greet(name, prefix="Hello"):
    return f"{prefix}, {name}"
print(greet("Ravi"))
```

Hands-on exercise: Create a configurable invoice function.

Day 18 - Lambda and higher-order functions

Concepts: lambda, sorted(key=...), map/filter concepts.

```
items = [("Pen", 20), ("Book", 90)]
print(sorted(items, key=lambda x: x[1]))
```

Hands-on exercise: Sort student records by marks.

Day 19 - Modules and packages

Concepts: import, from import, standard library math, random, datetime; pip and venv.

```
import random
print(random.randint(1, 6))
```

Hands-on exercise: Build a dice roller and date formatter.

Day 20 - Recursion and review

Concepts: Base cases, recursive calls and iterative alternatives.

```
def factorial(n):
    return 1 if n <= 1 else n * factorial(n-1)
print(factorial(5))
```

Hands-on exercise: Mini project: reusable math utility package.

Weekly checkpoint

Complete the mini project, explain your solution, and attempt a 10-question quiz. Target: at least 70% before moving on.

Week 5: Strings and Files

Learning sequence: concepts, guided code, independent lab, daily review.

Day 21 - String operations

Concepts: Slicing, immutability, split, join, strip, replace, case methods.

```
text = "  Python Course  "
print(text.strip().lower().replace(" ", "-"))
```

Hands-on exercise: Count vowels and reverse a string.

Day 22 - Formatting and validation

Concepts: f-strings, search, startswith, simple input validation.

```
email = "user@example.com"
print("@" in email and "." in email)
```

Hands-on exercise: Validate simple username rules.

Day 23 - Text files

Concepts: with open(), read, write, append, encoding and paths.

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Study Python\n")
```

Hands-on exercise: Create a persistent notes app.

Day 24 - CSV and JSON

Concepts: csv.reader/DictWriter, json.load/dump, serialization.

```
import json
data = {"course": "Python", "weeks": 8}
print(json.dumps(data, indent=2))
```

Hands-on exercise: Export student records as JSON.

Day 25 - File-based mini project

Concepts: Input validation, persistence, and structured records.

```
from pathlib import Path
p = Path("notes.txt")
print(p.exists())
```

Hands-on exercise: Mini project: contact book using JSON.

Weekly checkpoint

Complete the mini project, explain your solution, and attempt a 10-question quiz. Target: at least 70% before moving on.

Week 6: Object-Oriented Python

Learning sequence: concepts, guided code, independent lab, daily review.

Day 26 - Classes and objects

Concepts: Class definitions, __init__, self, attributes and methods.

```
class Student:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hi {self.name}"
print(Student("Asha").greet())
```

Hands-on exercise: Model a Course and Student.

Day 27 - Encapsulation

Concepts: Public/private conventions, properties and validation.

```
class Account:
    def __init__(self, balance=0):
        self._balance = balance
    @property
    def balance(self): return self._balance
```

Hands-on exercise: Add validated deposit and withdrawal methods.

Day 28 - Inheritance

Concepts: Parent/child classes, super(), method overriding.

```
class Person:
    def role(self): return "person"
class Mentor(Person):
    def role(self): return "mentor"
print(Mentor().role())
```

Hands-on exercise: Create Person, Student, and Mentor classes.

Day 29 - Polymorphism and composition

Concepts: Duck typing, shared interfaces, composition vs inheritance.

```
class EmailAlert:
    def send(self): return "email sent"
class SMSAlert:
    def send(self): return "sms sent"
for alert in [EmailAlert(), SMSAlert()]: print(alert.send())
```

Hands-on exercise: Design multiple payment methods.

Day 30 - OOP project

Concepts: Design objects, responsibilities and class relationships.

```
class Book:
    def __init__(self, title): self.title = title
    def __repr__(self): return f"Book({self.title!r})"
```

Hands-on exercise: Mini project: library management system.

Weekly checkpoint

Complete the mini project, explain your solution, and attempt a 10-question quiz. Target: at least 70% before moving on.

Week 7: Robust Python and Databases

Learning sequence: concepts, guided code, independent lab, daily review.

Day 31 - Exceptions

Concepts: try/except/else/finally, ValueError, custom error messages.

```
try:
    age = int("abc")
except ValueError:
    print("Enter a valid number")
```

Hands-on exercise: Harden the calculator against bad input.

Day 32 - Comprehensions

Concepts: List/dict/set comprehensions, readability and filtering.

```
squares = [n*n for n in range(1, 6)]
print(squares)
```

Hands-on exercise: Filter passed students from a list.

Day 33 - Iterators and generators

Concepts: iter(), next(), yield and lazy sequences.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
print(list(countdown(3)))
```

Hands-on exercise: Generate paginated records.

Day 34 - Decorators and testing

Concepts: Function wrappers, @decorator, assert and unittest basics.

```
def double(fn):
    def wrapper(x): return 2 * fn(x)
    return wrapper
@double
def increment(x): return x + 1
print(increment(3))
```

Hands-on exercise: Write unit tests for grade functions.

Day 35 - SQLite

Concepts: sqlite3 connection, parameterized queries, CREATE/INSERT/SELECT, commit.

```
import sqlite3
with sqlite3.connect(":memory:") as db:
    db.execute("CREATE TABLE students (name TEXT)")
    db.execute("INSERT INTO students VALUES (?)", ("Asha",))
    print(db.execute("SELECT * FROM students").fetchall())
```

Hands-on exercise: Mini project: SQLite student directory.

Weekly checkpoint

Complete the mini project, explain your solution, and attempt a 10-question quiz. Target: at least 70% before moving on.

Week 8: APIs and Final Project

Learning sequence: concepts, guided code, independent lab, daily review.

Day 36 - HTTP and APIs

Concepts: HTTP methods, status codes, JSON responses, timeouts and error handling.

```
import urllib.request
# Use a trusted test API endpoint when online.
# Always handle network errors and timeouts.
```

Hands-on exercise: Explain GET vs POST and parse sample JSON.

Day 37 - Project planning

Concepts: Requirements, menus, entities, database schema and task breakdown.

```
FEATURES = ["Add", "List", "Search", "Update", "Delete"]
print(FEATURES)
```

Hands-on exercise: Write project scope and flowchart.

Day 38 - Implementation

Concepts: Implement CRUD using functions/classes and SQLite parameter binding.

```
def validate_name(name):
    if not name.strip(): raise ValueError("Name required")
    return name.strip()
```

Hands-on exercise: Implement the core student management features.

Day 39 - Testing and polishing

Concepts: Unit tests, boundary cases, README, user guidance and code review.

```
import unittest
class Tests(unittest.TestCase):
    def test_basic(self): self.assertEqual(2 + 2, 4)
```

Hands-on exercise: Test invalid inputs, duplicates and missing records.

Day 40 - Demo and assessment

Concepts: Project demonstration, technical viva, review and next learning steps.

```
print("Final project ready for demonstration")
```

Hands-on exercise: Submit code, README, sample output and a 5-minute demo.

Weekly checkpoint

Complete the mini project, explain your solution, and attempt a 10-question quiz. Target: at least 70% before moving on.

Assessment Plan and Rubric

Weekly quizzes (8 x 10 marks): 20% of final grade

Weekly labs and mini projects: 30%

Final project functionality and code quality: 35%

Final demo, documentation and viva: 15%

Recommended passing criterion: 70% overall and submission of the final project.

Final Project: Student Course Management CLI

Build a command-line application with a menu to add students, create courses, enroll students, list enrollments, record marks, search records, and produce a summary. Persist records with SQLite, validate inputs, handle missing IDs gracefully, and include at least five automated tests.

Suggested SQLite tables

```
CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE courses (id INTEGER PRIMARY KEY, title TEXT NOT NULL);
CREATE TABLE enrollments (student_id INTEGER, course_id INTEGER, mark REAL,
  PRIMARY KEY (student_id, course_id));
```

Project submission checklist

Source files, requirements or standard-library note, README with run instructions, schema, test cases, screenshots of sample output, and a short demonstration.

Sample quiz questions

1. What type does input() return?

2. What is the difference between / and //?

3. When should elif be used?

4. How is a set different from a list?

5. What does a function return by default?

6. Why is with open() preferred?

7. What does self refer to in an instance method?

8. What is a ValueError?

9. Why should SQLite queries use ? placeholders?

10. What does an HTTP 404 status indicate?

Answer key

1. str; 2. true division vs floor division; 3. alternative condition; 4. unique unordered elements vs ordered sequence; 5. None; 6. automatic resource cleanup; 7. current instance; 8. invalid value for an operation; 9. safe parameter binding; 10. resource not found.

Instructor guidance

Use Python 3.11+ where possible. Adapt pacing to student proficiency. Demonstrate errors rather than hiding them. Require students to type and modify examples, not merely copy them. Encourage meaningful names, small functions, version control and incremental testing.